目录

1 杂项 2

1.1 头文件 2

1.2 i128 读写 2

1.3 差分 2

1.4 KMP 2

1.5 String Hash 3

1.6 背包 3

1.7 博弈 3

2 数学 4

2.1 快速幂 4

2.2 质因子分解 4

2.3 质数筛 5

2.4 逆元 5

2.5 组合数学 5

2.6 进制转换 6

2.7 二元一次不定方程 6

2.8 二项式反演 7

3 数据结构 7

3.1 并查集 7

3.2 单调队列 7

3.3 单调栈 8

3.4 懒删除堆 8

3.5 Trie 8

3.6 异或 Trie 9

3.7 树状数组 9

3.8 线段树 9

3.9 线段树 10

3.10 线段树 11

3.11 线段树 12

4 图论 14

4.1 链式前向星 14

4.2 拓扑排序 14

4.3 最小生成树 Kruskal 15

4.4 最小生成树 Prim 15

4.5 最短路 Floyd 15

4.6 最短路 Dijkstral 15

4.7 最短路 SPFA 16

4.8 LCA 16

5 近期做的一些题（可删） 16

5.1 轮廓线 DP 17

5.2 互不侵犯 17

5.3 分组过桥 17

5.4 用 t 拼 s, 求最短 t 18

5.5 能量项链 18

5.6 编辑距离 18

5.7 铁盘整理 IDA* 18

5.8 流星雨 19
5.9 青蛙的约会 19
5.10 生成字符串（卡特兰数） 20
5.11 反向建边求返程 20
5.12 网格图连通 20
5.13 二分判连通 20
5.14 最大三段和 21
5.15 质数表跳跃 21
5.16 任意段和不整除 22

1 杂项

1.1 头文件

```
#include<bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef unsigned long long ull;
typedef __int128 i128;

void solve(){
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int _=1;
    //cin>>_;
    cout<<fixed<<setprecision(15);
    while(_--)solve();
}
```

1.2 i128 读写

```
void read(i128&x){
    x=0;int f=1;charch=getchar();
    while(ch<'0'||ch>'9'){if(ch=='-')f=-1;ch=getchar()
        ;}
    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar()
        ;}
    x*=f;
}
void print(i128 x) {
    if(x<0){putchar('-');x=-x;}
    if(x>9)print(x/10);
    putchar(x%10+'0');
}
int main(){
    i128 a;
    read(a);
    print(a);
}
```

1.3 差分

```
一维差分
对于原始数组 a[]
通过 d[i]=a[i]-a[i-1] 初始化出 d[] 差分数组
对差分数组进行若干次修改

对区间 [l,r] 加 k
void add(int l,int r,int k){
    d[l]+=k;
    d[r+1]-=k;
}
最后累加得到结果
下标从 1 到 n
void up(int n){
    for(int i=1;i<=n;++i){
        d[i]+=d[i-1];
        a[i]=d[i];
    }
}

二维差分
为了避免若干边界问题
对于一个 n*m 的原始数组 a[n][m]
加工出一个 (n+2)*(m+2) 的差分数组 d[n+2][m+2]
最外层都用 0 包裹

对以[i,j]为左上角,[x,y]为右下角的矩形 + k
void add(int i,int j,int x,int y,int k){
    d[i][j]+=k;
    d[i][y+1]-=k;
    d[x+1][j]-=k;
    d[x+1][y+1]+=k;
}
累加操作类比二维前缀和
void up(int n,int m){
    for(int i=1;i<=n;++i){
        for(int j=1;j<=m;++j){
            d[i][j]=d[i-1][j]+d[i][j-1]-d[i-1][j-1];
        }
    }
}
等差数列差分
区间[l,r]加上一个以s为首项d为公差e为末项的等差数列
void add(int l,int r,int s,int d,int e){
    a[l]+=s;
    a[l+1]+=d-s;
    a[r+1]+=-d-e;
    a[r+2]+=e;
}
累加两次
void up(int n){
    for(int i=1;i<=n;++i){
        a[i]+=a[i-1];
    }
    for(int i=1;i<=n;++i){
        a[i]+=a[i-1];
    }
}
```

1.4 KMP

至今不会 KMP

$\pi[i]$ 表示子串 $s[0\ldots i]$ 最长的相等真前后缀长度。

使用 0-indexed 写法

```
//返回pattern在text中所有出现位置的起始索引
vector<int> kmp(string&text,string&pattern){
    int n=text.length();
    int m =pattern.length();
    if (m == 0) return {};
    //1.求解 pi 数组
    vector<int>pi(m);
    for (int i=1,j=0;i<m;i++){
        while(j&&pattern[i]!=pattern[j])j=pi[j-1];
        if(pattern[i]==pattern[j])j++;
        pi[i]=j;
    }
```

```
    //2.字符串匹配
    //记录匹配的起始起点
    vector<int> res;
    for(int i=0,j=0;i<n;i++){
        while(j&&text[i]!=pattern[j])j=pi[j-1];
        if(text[i]==pattern[j])j++;
        if(j==m){
            res.push_back(i-m+1);
            j=pi[j-1];
        }
    }
    return res;
}
```

1.5 String Hash

1-indexed

```
const int N = 1e6 + 5;
const int P = 131;
ull h[N], p[N];
void init_hash(const string& s) {
    p[0] = 1;
    for (int i = 1; i <= s.length(); ++i) {
        p[i] = p[i - 1] * P;
        h[i] = h[i - 1] * P + s[i - 1];
    }
}
// O(1)查询原串中 [l, r] 区间的哈希值
ull get_hash(int l, int r) {
    return h[r] - h[l - 1] * p[r - l + 1];
}
```

1.6 背包

```
// W: 背包总容量, N: 物品数量
// weight[i], value[i]: 第 i 个物品的重量和价值
vector<long long> dp(W + 1, 0);
// 01 背包
for (int i = 0; i < N; ++i) {
    for (int j = W; j >= weight[i]; --j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i
            ]);
    }
}
// 完全背包
for (int i = 0; i < N; ++i) {
    for (int j = weight[i]; j <= W; ++j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i
            ]);
    }
}
// 多重背包
struct Item { int w, v; };
vector<Item> items;
// count[i] 表示第 i 个物品的数量
for (int i = 0; i < N; ++i) {
    int c = count[i];
    for (int k = 1; k <= c; k <<= 1) {
        items.push_back({weight[i] * k, value[i] * k});
        c -= k;
    }
    if (c > 0) {
        items.push_back({weight[i] * c, value[i] * c});
    }
}
for (auto& item : items) {
    for (int j = W; j >= item.w; --j) {
        dp[j] = max(dp[j], dp[j - item.w] + item.v);
    }
}
```

1.7 博弈

```
巴氏博弈

一堆, n 个石子, 每次取[1,m]个, 若n%(m+1)==0, 先手必败

Nim Game

n 堆石子, 每次选 1 堆拿任意个

a1 XOR a2 XOR……XOR an == 0 先手必败

斐波那契博弈

1 堆石子, 共n个。先手第一次不能拿完。之后每次拿的数量,
    最多是对手上一次拿的2倍。

如果 n 是一个斐波那契数, 先手必败

sg 函数

sg 值为 0 先手必败

多局将每局 sg 异或

const int N = 10005;
int f[N];// 记忆化数组, 初始化为 -1
int s[N], k;// 允许拿取的集合 S, 大小为 k

void init_sg() {
    memset(f, -1, sizeof(f));
    f[0] = 0; // 终局状态无法操作, SG必定为0
}

int sg(int x) {
    if (f[x] != -1) return f[x];

    // 使用 unordered_set 或 bool 数组记录后继状态的 SG
        值
    bool g[N] = {0};

    // 遍历所有合法的拿法
    for (int i = 0; i < k; ++i) {
        if (x >= s[i]) {
            g[sg(x - s[i])] = 1;
        }
    }

    // 求 MEX (最小未出现的非负整数)
    for (int i = 0; ; ++i) {
        if (!g[i]) return f[x] = i;
```

```
    }
}
```

2 数学

2.1 快速幂

```
ll qpow(ll b,ll e){
    ll res=1;
    while(e){
        if(e&1)res*=b;
        b*=b;
        e>>=1;
    }
    return res;
}

ll qpow(ll b,ll e,ll MOD){
    b%=MOD;
    ll res=1;
    while(e){
        if(e&1)res=res*b%MOD;
        b=b*b%MOD;
        e>>=1;
    }
    return res;
}

i128 mul(i128 a,i128 b,i128 MOD){
    a%=MOD;
    i128 res=0;
    while(b){
        if(b&1)res=(res+a)%MOD;
        a=(a<<1)%MOD;
        b>>=1;
    }
    return res;
}
i128 qpow(i128 b,i128 e,i128 MOD){
    b%=MOD;
    i128 ans=1;
    while(e){
        if(e&1)ans=mul(ans,b,MOD);
        b=mul(b,b,MOD);
        e>>=1;
    }
    return ans;
}
```

2.2 质因子分解

得到 n 的所有质因子

```
vector<int>funtion(int n){
    vector<int>p;
    for(int i=2;i*i<=n;++i){
        if(n%i==0){
            p.push_back(i);
            while(n%i==0)n/=i;
        }
    }
    if(n>1)p.push_back(n);
    return p;
}
```

2.3 质数筛

```
const int N=1e5;
bitset<N>isp;
vector<int>p;
void ehrlich(int n){
    isp.set();
    isp[0]=isp[1]=0;
    for(int i=2;i*i<=n;++i){
        if(isp[i]){
            p.push_back(i);
            for(int j=i*i;j<=n;j+=i){
                isp[j]=0;
            }
        }
    }
}
void euler(int n){
    isp.set();
    isp[0]=isp[1]=0;
    for(int i=2;i<=n;++i){
        if(isp[i])p.push_back(i);
        for(int x:p){
            if(x*i>n)break;
            isp[x*i]=0;
            if(i%x==0)break;
        }
    }
}
仅记数
int ehrlich_cnt(int n){
    if(n<=1)return 0;
    vector<int>isp(n+1,1);
    int cnt=(n+1)/2;
    for(int i=3;i*i<=n;i+=2){
        if(isp[i]){
            for(int j=i*i;j<=n;j+=2*i){
                if(isp[j]){
                    isp[j]=0;
                    cnt--;
                }
            }
        }
    }
    return cnt;
}
```

2.4 逆元

求 X 在 mod(MOD) 意义下的逆元

```
typedef long long ll;
const int MOD=1e9+7;
费马小定理
要求 MOD 为质数
ll qMODow(ll b,ll e,ll MOD){
    ll res=1;
    b%=MOD;
    while(e){
        if(e&1)res=res*b%MOD;
        e>>=1;
        b=b*b%MOD;
    }
    return res;
}
ll inv(ll X,ll MOD){
    return qMODow(X,MOD-2,MOD);
}
拓展欧几里得
要求 x 与 MOD 互质
int ex_gcd(int a,int b,int &x,int &y){
    if(b==0){
        x=1,y=0;
        return a;
    }
    int g=ex_gcd(b,a%b,y,x);
    y-=a/b*x;
    return g;
}
int inv(int X,int p){
    int x,y;
    ex_gcd(X,p,x,y);
    return (x%p+p)%p;
}
线性处理逆元
const int N=1e8+5;
int inv[N];
void init(int n,int p){
    inv[1]=1;
    for(int i=2;i<=n;++i){
        inv[i]=1LL*(p-p/i)*inv[p%i]%p;
    }
}
```

2.5 组合数学

```
const int N=1e6+5;
const int MOD=1e9+7;
ll qpow(ll b,ll e){
    ll res=1;
    while(e){
        if(e&1)res=res*b%MOD;
        b=b*b%MOD;
        e>>=1;
    }
    return res;
}
ll f[N],inv[N];
void init(){
    f[0]=f[1]=1;
    for(int i=2;i<N;++i){
        f[i]=i*f[i-1]%MOD;
    }
    inv[N-1]=qpow(f[N-1],MOD-2);
    for(int i=N-1;i;--i){
        inv[i-1]=i*inv[i]%MOD;
    }
}
排列数
ll A(int n,int m){
    if(n<0||m<0||n<m)return 0;
    return f[n]*inv[n-m]%MOD;
}
```

```
组合数
ll C(int n,int m){
    if(n<0||m<0||n<m)return 0;
    return f[n]*inv[m]%MOD*inv[n-m]%MOD;
}
隔板 将n个元素分成m份(每份至少一个元素)
ll C_1(int n,int m){
    if(m==0)return n==0?1:0;
    if(n<m)return 0;
    return C(n-1,m-1);
}
```

2.6 进制转换

```
namespace X_to_Y{
    char dtoc(int d){
        if(d>9)return 'A'+d-10;
        return '0'+d;
    }
    int ctod(char c){
        if(c>'a')return c-'a'+10;
        if(c>='A')return c-'A'+10;
        return c-'0';
    }
    ll pos_to_ten(string num,int x){
        ll t=0;
        for(char c:num){
            t=t*x+ctod(c);
        }
        return t;
    }
    ll neg_to_ten(string num,int x) {
        ll t=0;
        ll b=1;
        for(int i=num.size()-1;i>=0;i--){
            t+=ctod(num[i])*b;
            b*=x;
        }
        return t;
    }
    string ten_to_y(ll t,int y){
        if(t==0)return "0";
        string ans="";
        while(t){
            int d=t%y;
            t/=y;
            if(d<0){
                d+=-y;
                t++;
            }
            ans=dtoc(d)+ans;
        }
        return ans;
    }
    string x_to_y(string num,int x,int y){
        if(num=="0")return num;
        ll t=(x<0)?neg_to_ten(num,x):pos_to_ten(num,x);
        return ten_to_y(t,y);
    }
}using namespace X_to_Y;
```

2.7 二元一次不定方程

```
裴蜀定理

对于不全为零的整数 a,b

一定存在一组 x,y

使得 ax + by = gcd(a,b)

推论:

ax + by = 1 有解

当且仅当, gcd(a,b)=1, 即 a,b 互质

ax + by = c 有解

当且仅当, gcd(a,b) | c, 即 c 是 gcd(a,b) 的倍数

可由两项推广到多项

拓展欧几里得定理

解 ax + by = gcd(a,b)

返回值为gcd(a, b)

同时求出 ax+by=gcd(a,b) 的一组解 x,y

int ex_gcd(int a,int b,int &x,int &y){
    if(b==0){
        x=1,y=0;
        return a;
    }
    int g=ex_gcd(b,a%b,y,x);
    y-=a/b*x;
    return g;
}

二元一次方程

解 ax + by = c

一、有解判定

根据裴蜀定理: 当且仅当 gcd(a,b) | c 时有解

二、特解 X,Y

根据拓展欧几里得定理:

先求 x, y : ax + by = gcd(a,b)

X = x * c/gcd(a,b)

Y = y * c/gcd(a,b)

三、通解

参数 kx,ky :
```

```

kx = b / gcd(a,b)

ky = a / gcd(a,b)

通解 x,y:

x = X + k \* kx

y = Y - k \* ky

四、最小正整数解

x = ( X % kx + kx ) % kx
```

2.8 二项式反演

“求恰好满足 k 个条件的方案数”困难

但“求强制满足 k 个条件（其余随意）的方案数”容易

```
设一共有n个条件

g(i)表示 恰好 满足i个条件的方案数

f(k)表示 至少 满足k个条件的方案数（选k个条件强制满足，剩下的n-k个随意）

vector<long long>f(n+1);

long long ans=0;

for(int i=k;i<=n;++i){
    long long term=C(i,k)*f[i]%MOD;
    if((i-k)&1){
        ans=(ans-term+MOD)%MOD;
    }
    else{
        ans=(ans+term)%MOD;
    }
}
```

3 数据结构

3.1 并查集

```
template<class>
class UF{
    vector<int>siz,fa;
    int cnt;
public:
    UF(int n):cnt(n),fa(n),siz(n){
        for(int i=0;i<n;++i){
            fa[i]=i;
            siz[i]=1;
        }
    }
    int find(int x){
        return x==fa[x]?x:fa[x]=find(fa[x]);
    }
    bool check(int a,int b){
        return find(a)==find(b);
    }
    bool merge(int a,int b){
        a=find(a);
        b=find(b);
        if(a==b)return 0;
        if(siz[a]>siz[b]){
            fa[b]=a;
            siz[a]+=siz[b];
        }
        else{
            fa[a]=b;
            siz[b]+=siz[a];
        }
        cnt--;
        return 1;
    }
    int count(){
        return cnt;
    }
    int size(int x){
        return siz[find(x)];
    }
};
```

3.2 单调队列

大常数

```
template<typename T>
class MonotonicQueue{
    deque<T>q,m,M;
public:
    void push(T e){
        q.push_back(e);
        while(!m.empty()&&m.back()>e)m.pop_back();
        m.push_back(e);
        while(!M.empty()&&M.back()<e)M.pop_back();
        M.push_back(e);
    }
    T max(){return M.front();}
    T min(){return m.front();}
    T pop(){
```

```
        T x=q.front();
        q.pop_front();
        if(x==m.front())m.pop_front();
        if(x==M.front())M.pop_front();
        return x;
    }
    int size()const{return q.size();}
    bool empty()const{return q.size()==0;}
};
```

3.3 单调栈

获取每一个元素两侧第一个大于（小于）它的元素

```
const int N=1e5;
int a[N];
int top,st[N],L[N],R[N];
int n;
void init(){
    top=-1;
    for(int r=0;r<n;++r){
        // "<=" 获取第一个大于a[i]的元素
        // ">=" 获取第一个小于a[i]的元素
        while(top!=-1&&a[st[top]]<=a[r]){
            int i=st[top--];
            L[i]=top==-1?-1:st[top];
            R[i]=r;
        }
        st[++top]=r;
    }
    while(top!=-1){
        int i=st[top--];
        L[i]=top==-1?-1:st[top];
        R[i]=n;
    }
    //处理重复元素
    for(int i=n-2;i>=0;--i){
        if(R[i]!=n&&a[R[i]]==a[i]){
            R[i]=R[R[i]];
        }
    }
}
```

3.4 懒删除堆

```
template<typename T,typename cmp=less<T>>
    class LazyHeap{
        priority_queue<T,vector<T>,cmp>pq;
        unordered_map<T,int>tag;
        size_t siz=0;//真实大小
        void up(){
            while(pq.size()&&tag[pq.top()]>0){
                tag[pq.top()]--;
                pq.pop();
            }
        }
    public:
        size_t size(){
            return siz;
        }
        void erase(T x){//懒删除
            tag[x]++;
            siz--;
        }
        T top(){
            up();
            return pq.top();
        }
        void pop(){
            up();
            pq.pop();
            siz--;
        }
        void push(T x){
            pq.push(x);
            siz++;
        }
    };
```

3.5 Trie

支持数字，大小写字母

```
class Trie{
    private:
    static const int N=1e7+5;
    int T[N][26+26+10];
    int pass[N];
    int end[N];
    int cnt=1;
    int get_index(char x){
        if(x>='0'&&x<='9')return x-'0';
        if(x>='a'&&x<='z')return x-'a'+10;
        return x-'A'+36;
    }
public:
    void add(string word){
        int cur=1;
        pass[cur]++;
        for(char x:word){
            int nex=get_index(x);
            if(T[cur][nex]==0){
                T[cur][nex]=++cnt;
            }
            cur=T[cur][nex];
            pass[cur]++;
        }
        end[cur]++;
    }
    int search(string word){
        int cur=1;
        for(char x:word){
            int nex=get_index(x);
            if(T[cur][nex]==0)return 0;
            cur=T[cur][nex];
        }
        return end[cur];
    }
    int pre_cnt(string pre){
        int cur=1;
        for(char x:pre){
            int nex=get_index(x);
            if(T[cur][nex]==0)return 0;
            cur=T[cur][nex];
```

```
        }
        return pass[cur];
    }
    void remove(string word){
        if(search(word)==0)return;
        int cur=1;
        for(char x:word){
            int nex=get_index(x);
            if(--pass[T[cur][nex]]==0){
                T[cur][nex]=0;
                return;
            }
            cur=T[cur][nex];
        }
        end[cur]--;
    }
    void clean(){
        for(int i=1;i<=cnt;++i){
            pass[i]=0;
            end[i]=0;
            memset(T[i],0,sizeof(T[0]));
        }
        cnt=1;
    }
};
```

3.6 异或 Trie

add(x) 向集合中添加元素

del(x) 从集合中删除元素

query(x) y 属于集合，查询 x XOR y 的最大值

```
template<typename T>
class Trie{
    int N;
    struct node{
        int p;
        vector<node*>nex;
        node():p(0),nex(2,nullptr){}
    };
    node*root;
public:
    Trie(int n):N(n){
        root=new node();
    }
    void add(T x){
        node*cur=root;
        for(int i=N-1;i>=0;--i){
            int j=(x>>i)&1;
            if(cur->nex[j]==nullptr)cur->nex[j]=new node();
            cur=cur->nex[j];
            cur->p++;
        }
    }
    void del(T x){
        node*cur=root;
        for(int i=N-1;i>=0;--i){
            int j=(x>>i)&1;
            cur=cur->nex[j];
            cur->p--;
        }
    }
    T query(T x){
        T y=0;
        node*cur=root;
        for(int i=N-1;i>=0;--i){
            int u=(x>>i)&1;
            int v=u^1;
            if(cur->nex[v]&&cur->nex[v]->p){
                y|=(v<<i);
                cur=cur->nex[v];
            }
            else{
                y|=(u<<i);
                cur=cur->nex[u];
            }
        }
        return x^y;
    }
};
```

3.7 树状数组

单点修改，区间查询

索引从 1 开始。

```
struct BIT{
    int n;
    vector<ll> tree;
    BIT(int _n):n(_n),tree(_n+1,0){}
    int lowbit(int x){return x&(-x);}
    //单点修改:a[x]+=k
    void add(int x,ll k){
        for(;x<=n;x+=lowbit(x))tree[x]+=k;
    }
    //前缀和查询:求a[1]到a[x]的和
    ll query(int x){
        ll res=0;
        for(;x>0;x-=lowbit(x))res+=tree[x];
        return res;
    }
    //区间查询:求a[L]到a[R]的和
    ll query(int L,int R){
        return query(R)-query(L-1);
    }
};
```

不知道怎么整理线段树的板子

3.8 线段树

区间加，区间乘，查询区间和

```
const int N=1e5+5;
const int P;// MOD
ll a[N];
ll sum[N<<2];
ll add[N<<2];
ll mul[N<<2];
void up(int x){
    sum[x]=(sum[x<<1]+sum[x<<1|1])%P;
}
void built(int x,int l,int r){
```

```
    if(l==r){
        sum[x]=a[l]%P;
    }
    else{
        int mid=(l+r)>>1;
        built(x<<1,l,mid);
        built(x<<1|1,mid+1,r);
        up(x);
    }
    add[x]=0;
    mul[x]=1;
}
void lazy_add(int x,ll k,int siz){
    sum[x]=(sum[x]+k*siz)%P;
    add[x]=(add[x]+k)%P;
}
void lazy_mul(int x,int k){
    sum[x]=sum[x]*k%P;
    mul[x]=mul[x]*k%P;
    add[x]=add[x]*k%P;
}
void down(int x,int lsiz,int rsiz){
    if(mul[x]!=1){
        lazy_mul(x<<1,mul[x]);
        lazy_mul(x<<1|1,mul[x]);
        mul[x]=1;
    }
    if(add[x]){
        lazy_add(x<<1,add[x],lsiz);
        lazy_add(x<<1|1,add[x],rsiz);
        add[x]=0;
    }
}
void MUL(int x,int l,int r,int ql,int qr,ll k){
    if(ql<=l&&r<=qr){
        lazy_mul(x,k);
        return;
    }
    int mid=(l+r)>>1;
    down(x,mid-l+1,r-mid);
    if(ql<=mid)MUL(x<<1,l,mid,ql,qr,k);
    if(qr>mid)MUL(x<<1|1,mid+1,r,ql,qr,k);
    up(x);
}
void ADD(int x,int l,int r,int ql,int qr,ll k){
    if(ql<=l&&r<=qr){
        lazy_add(x,k,r-l+1);
        return;
    }
    int mid=(l+r)>>1;
    down(x,mid-l+1,r-mid);
    if(ql<=mid)ADD(x<<1,l,mid,ql,qr,k);
    if(qr>mid)ADD(x<<1|1,mid+1,r,ql,qr,k);
    up(x);
}
int query(int x,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr){
        return sum[x];
    }
    int mid=(l+r)>>1;
    down(x,mid-l+1,r-mid);
    int ans=0;
    if(ql<=mid)ans=(ans+query(x<<1,l,mid,ql,qr))%P;
    if(qr>mid)ans=(ans+query(x<<1|1,mid+1,r,ql,qr))%P;
    return ans;
}
```

3.9 线段树

区间加，区间覆盖，查询区间最大值

```
const int N=1e6+5;
ll a[N];
ll Max[N<<2];
ll add[N<<2];
ll cha[N<<2];
bool need[N<<2];
void up(int x){
    Max[x]=max(Max[x<<1],Max[x<<1|1]);
}
void built(int x,int l,int r){
    if(l==r){
        Max[x]=a[l];
    }
    else{
        int mid=(l+r)>>1;
        built(x<<1,l,mid);
        built(x<<1|1,mid+1,r);
        up(x);
    }
    add[x]=0;
    need[x]=0;
}
void lazy_add(int x,ll k){
    Max[x]+=k;
    add[x]+=k;
}
void lazy_change(int x,ll k){
    add[x]=0;
    Max[x]=k;
    need[x]=1;
    cha[x]=k;
}
void down(int x){
    if(need[x]){
        lazy_change(x<<1,cha[x]);
        lazy_change(x<<1|1,cha[x]);
        need[x]=0;
    }
    if(add[x]){
        lazy_add(x<<1,add[x]);
        lazy_add(x<<1|1,add[x]);
        add[x]=0;
    }
}
ll query(int x,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr)return Max[x];
    int mid=(l+r)>>1;
    down(x);
    ll ans=LONG_LONG_MIN;
    if(ql<=mid)ans=max(ans,query(x<<1,l,mid,ql,qr));
    if(qr>mid)ans=max(ans,query(x<<1|1,mid+1,r,ql,qr));
    return ans;
```

```
}
void change(int x,int l,int r,int ql,int qr,ll k){
    if(ql<=l&&r<=qr){
        lazy_change(x,k);
        return;
    }
    int mid=(l+r)>>1;
    down(x);
    if(ql<=mid)change(x<<1,l,mid,ql,qr,k);
    if(qr>mid)change(x<<1|1,mid+1,r,ql,qr,k);
    up(x);
}
void ADD(int x,int l,int r,int ql,int qr,ll k){
    if(ql<=l&&r<=qr){
        lazy_add(x,k);
        return;
    }
    int mid=(l+r)>>1;
    down(x);
    if(ql<=mid)ADD(x<<1,l,mid,ql,qr,k);
    if(qr>mid)ADD(x<<1|1,mid+1,r,ql,qr,k);
    up(x);
}
void solve(){
    int n,m,op,l,r;
    ll k;
    cin>>n>>m;
    for(int i=1;i<=n;++i)cin>>a[i];
    built(1,1,n);
    while(m--){
        cin>>op;
        if(op==1){
            cin>>l>>r>>k;
            change(1,1,n,l,r,k);
        }
        if(op==2){
            cin>>l>>r>>k;
            ADD(1,1,n,l,r,k);
        }
        if(op==3){
            cin>>l>>r;
            cout<<query(1,1,n,l,r)<<'\n';
        }
    }
}
```

3.10 线段树

新生赛打卡题

01 线段树，区间和并

```
const int N=1e5+10;
int a[N];
int sum[N<<2];
int L0[N<<2];
int R0[N<<2];
int M0[N<<2];
int L1[N<<2];
int R1[N<<2];
int M1[N<<2];
int tag1[N<<2];
int tag2[N<<2];
void up(int x,int l,int r){
    int mid=(l+r)>>1;
    sum[x]=sum[x<<1]+sum[x<<1|1];
    L0[x]=L0[x<<1];
    if(M0[x<<1]==mid-l+1)L0[x]+=L0[x<<1|1];
    L1[x]=L1[x<<1];
    if(M1[x<<1]==mid-l+1)L1[x]+=L1[x<<1|1];
    R0[x]=R0[x<<1|1];
    if(M0[x<<1|1]==r-mid)R0[x]+=R0[x<<1];
    R1[x]=R1[x<<1|1];
    if(M1[x<<1|1]==r-mid)R1[x]+=R1[x<<1];
    M0[x]=max({M0[x<<1],M0[x<<1|1],R0[x<<1]+L0[x
        <<1|1]});
    M1[x]=max({M1[x<<1],M1[x<<1|1],R1[x<<1]+L1[x
        <<1|1]});
}
void lazy_cover(int x,int l,int r,int val){
    tag1[x]=val;
    tag2[x]=0;
    if(val==1){
        sum[x]=L1[x]=R1[x]=M1[x]=r-l+1;
        L0[x]=R0[x]=M0[x]=0;
    }
    else{
        sum[x]=L1[x]=R1[x]=M1[x]=0;
        L0[x]=R0[x]=M0[x]=r-l+1;
    }
}
void lazy_re(int x,int l,int r){
    if(tag1[x]!=-1){
        tag1[x]^=1;
    }
    else{
        tag2[x]^=1;
    }
    sum[x]=r-l+1-sum[x];
    swap(L1[x],L0[x]);
    swap(R1[x],R0[x]);
    swap(M1[x],M0[x]);
}
void down(int x,int l,int r){
    int mid=(l+r)>>1;
    if(tag1[x]!=-1){
        lazy_cover(x<<1,l,mid,tag1[x]);
        lazy_cover(x<<1|1,mid+1,r,tag1[x]);
        tag1[x]=-1;
    }
    if(tag2[x]){
        lazy_re(x<<1,l,mid);
        lazy_re(x<<1|1,mid+1,r);
        tag2[x]=0;
    }
}
void built(int x,int l,int r){
    tag1[x]=-1;
    tag2[x]=0;
    if(l==r){
        sum[x]=a[l];
        L0[x]=R0[x]=M0[x]=1-a[l];
        L1[x]=R1[x]=M1[x]=a[l];
    }
```

```
    else{
        int mid=(l+r)>>1;
        built(x<<1,l,mid);
        built(x<<1|1,mid+1,r);
        up(x,l,r);
    }
}
void cover(int x,int l,int r,int ql,int qr,int val){
    if(ql<=l&&r<=qr){
        lazy_cover(x,l,r,val);
        return;
    }
    down(x,l,r);
    int mid=(l+r)>>1;
    if(ql<=mid)cover(x<<1,l,mid,ql,qr,val);
    if(qr>mid)cover(x<<1|1,mid+1,r,ql,qr,val);
    up(x,l,r);
}
void re(int x,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr){
        lazy_re(x,l,r);
        return;
    }
    down(x,l,r);
    int mid=(l+r)>>1;
    if(ql<=mid)re(x<<1,l,mid,ql,qr);
    if(qr>mid)re(x<<1|1,mid+1,r,ql,qr);
    up(x,l,r);
}
int query3(int x,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr){
        return sum[x];
    }
    down(x,l,r);
    int res=0;
    int mid=(l+r)>>1;
    if(ql<=mid)res+=query3(x<<1,l,mid,ql,qr);
    if(qr>mid)res+=query3(x<<1|1,mid+1,r,ql,qr);
    return res;
}
int query4(int x,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr){
        return M1[x];
    }
    down(x,l,r);
    int mid=(l+r)>>1;
    if(ql>mid)return query4(x<<1|1,mid+1,r,ql,qr);
    if(qr<=mid)return query4(x<<1,l,mid,ql,qr);
    int res=max(query4(x<<1,l,mid,ql,qr),query4(x<<1|1,
        mid+1,r,ql,qr));
    int L=min(R1[x<<1],mid-ql+1);
    int R=min(L1[x<<1|1],qr-mid);
    return max(res,L+R);
}
```

3.11 线段树

矩形面积并，矩形周长并

```
面积
const int N=3e5+5;
struct rectangle{
    int x1,y1,x2,y2;
}rec[N/2];
struct lines{
    int x,y1,y2,k;
}L[N];
int Ysort[N];
int total[N<<2];
int cover[N<<2];
int cnt[N<<2];

void up(int x){
    if(cnt[x]>0)cover[x]=total[x];
    else cover[x]=cover[x<<1]+cover[x<<1|1];
}
void built(int x,int l,int r){
    if(l<r){
        int mid=(l+r)>>1;
        built(x<<1,l,mid);
        built(x<<1|1,mid+1,r);
    }
    total[x]=Ysort[r+1]-Ysort[l];
    cover[x]=0;
    cnt[x]=0;
}
void change(int x,int l,int r,int ql,int qr,int k){
    if(ql<=l&&r<=qr){
        cnt[x]+=k;
    }else{
        int mid=(l+r)>>1;
        if(ql<=mid)change(x<<1,l,mid,ql,qr,k);
        if(qr>mid)change(x<<1|1,mid+1,r,ql,qr,k);
    }
    up(x);
}
int init(int n){
    for(int i=1,j=i+n;i<=n;++i,++j){
        L[i]={rec[i].x1,rec[i].y1,rec[i].y2,1};
        L[j]={rec[i].x2,rec[i].y1,rec[i].y2,-1};
        Ysort[i]=rec[i].y1;
        Ysort[j]=rec[i].y2;
    }
    int siz=n<<1;
    sort(L+1,L+siz+1,[&](auto a,auto b){return a.x<b.x
        ;});
    sort(Ysort+1,Ysort+siz+1);
    int m=unique(Ysort+1,Ysort+siz+1)-(Ysort+1);
    Ysort[m+1]=Ysort[m];
    return m;
}
ll merge_S(int n){
    int m=init(n);
    built(1,1,m);
    ll ans=0;
    int pre=0;
    int siz=n<<1;
```

```
    for(int i=1;i<=siz;i++){
        ans+=1LL*cover[1]*(L[i].x-pre);
        pre=L[i].x;
        int l=lower_bound(Ysort+1,Ysort+m+1,L[i].y1)-Ysort;
        int r=lower_bound(Ysort+1,Ysort+m+1,L[i].y2)-Ysort-1;
        change(1,1,m,l,r,L[i].k);
    }
    return ans;
}
周长
const int N=1e5+5;
struct rectangle{
    int x1,y1,x2,y2;
}rec[N/2];
struct lines_y{
    int x,y1,y2,k;
}Ly[N];
struct lines_x{
    int y,x1,x2,k;
}Lx[N];
int Ysort[N],Xsort[N];
int total[N<<2];
int cover[N<<2];
int cnt[N<<2];

void up(int x){
    if(cnt[x]>0)cover[x]=total[x];
    else cover[x]=cover[x<<1]+cover[x<<1|1];
}
void change(int x,int l,int r,int ql,int qr,int k){
    if(ql<=l&&r<=qr){
        cnt[x]+=k;
    }else{
        int mid=(l+r)>>1;
        if(ql<=mid)change(x<<1,l,mid,ql,qr,k);
        if(qr>mid)change(x<<1|1,mid+1,r,ql,qr,k);
    }
    up(x);
}
void built_y(int x,int l,int r){
    if(l<r){
        int mid=(l+r)>>1;
        built_y(x<<1,l,mid);
        built_y(x<<1|1,mid+1,r);
    }
    total[x]=Ysort[r+1]-Ysort[l];
    cover[x]=0;
    cnt[x]=0;
}
int init_y(int n){
    for(int i=1,j=i+n;i<=n;++i,++j){
        Ly[i]={rec[i].x1,rec[i].y1,rec[i].y2,1};
        Ly[j]={rec[i].x2,rec[i].y1,rec[i].y2,-1};
        Ysort[i]=rec[i].y1;
        Ysort[j]=rec[i].y2;
    }
    int siz=n<<1;
    sort(Ly+1,Ly+siz+1,[&](auto a,auto b){return a.x<b.x;});
    sort(Ysort+1,Ysort+siz+1);
    int m=unique(Ysort+1,Ysort+siz+1)-(Ysort+1);
    Ysort[m+1]=Ysort[m];
    return m;
}
ll scan_y(int n){
    int m=init_y(n);
    built_y(1,1,m);
    ll ans=0;
    int pre;
    int siz=n<<1;
    for(int i=1;i<=siz;i++){
        pre=cover[1];
        int l=lower_bound(Ysort+1,Ysort+m+1,Ly[i].y1)-Ysort;
        int r=lower_bound(Ysort+1,Ysort+m+1,Ly[i].y2)-Ysort-1;
        change(1,1,m,l,r,Ly[i].k);
        ans+=abs(pre-cover[1]);
    }
    return ans;
}
void built_x(int x,int l,int r){
    if(l<r){
        int mid=(l+r)>>1;
        built_x(x<<1,l,mid);
        built_x(x<<1|1,mid+1,r);
    }
    total[x]=Xsort[r+1]-Xsort[l];
    cover[x]=0;
    cnt[x]=0;
}
int init_x(int n){
    for(int i=1,j=i+n;i<=n;++i,++j){
        Lx[i]={rec[i].y1,rec[i].x1,rec[i].x2,1};
        Lx[j]={rec[i].y2,rec[i].x1,rec[i].x2,-1};
        Xsort[i]=rec[i].x1;
        Xsort[j]=rec[i].x2;
    }
    int siz=n<<1;
    sort(Lx+1,Lx+siz+1,[&](auto a,auto b){return a.y<b.y;});
    sort(Xsort+1,Xsort+siz+1);
    int m=unique(Xsort+1,Xsort+siz+1)-(Xsort+1);
    Xsort[m+1]=Xsort[m];
    return m;
}
ll scan_x(int n){
    int m=init_x(n);
    built_x(1,1,m);
    ll ans=0;
    int pre;
    int siz=n<<1;
    for(int i=1;i<=siz;i++){
        pre=cover[1];
        int l=lower_bound(Xsort+1,Xsort+m+1,Lx[i].x1)-Xsort;
        int r=lower_bound(Xsort+1,Xsort+m+1,Lx[i].x2)-Xsort-1;
        change(1,1,m,l,r,Lx[i].k);
        ans+=abs(pre-cover[1]);
```

```
    }
    return ans;
}
ll merge_C(int n){
    return scan_x(n)+scan_y(n);
}
```

4 图论

4.1 链式前向星

```
const int N=1e5;//点
const int M=1e6;//边
int tot;
int head[N];
int nex[M];
int to[M];
T weight[M];
void init(int n){
    tot=0;
    memset(head,-1,sizeof(head));
}
void addEdge(int u,int v,T w){
    nex[tot]=head[u];
    to[tot]=v;
    weight[tot]=w;
    head[u]=tot++;
}
// 遍历从 u 出发的边
for(int ei=head[u];ei!=-1;ei=nex[ei]){
    int v=to[ei];
    T w=weight[ei];
}
```

4.2 拓扑排序

有向无环图

```
const int N=1e5;
vector<int>gra[N];
int in[N];
void top(int n){
    for(int u=0;u<n;++u){
        for(int v:gra[u]){
            in[v]++;
        }
    }
    queue<int>q;
    for(int i=0;i<n;++i){
        if(in[i]==0){
            q.push(i);
        }
    }
    while(q.size()){
        int u=q.front();
        q.pop();
        for(int v:gra[u]){
            if(--in[v]==0){
                q.push(v);
            }
        }
    }
}
```

4.3 最小生成树 Kruskal

```
class UF{
    vector<int>fa;
    int cnt;
public:
    UF(int n):fa(n),cnt(n){
        for(int i=0;i<n;++i){
            fa[i]=i;
        }
    }
    int find(int x){
        return fa[x]==x?x:fa[x]=find(fa[x]);
    }
    bool merge(int a,int b){
        a=find(a);
        b=find(b);
        if(a==b)return 0;
        fa[a]=b;
        cnt--;
        return 1;
    }
    int count(){
        return cnt;
    }
};
struct edge{
    int u,v;
    T w;
};
T kruskal(int n,vector<edge>&Edges){
    UF st(n);
    T sum=0;
    for(auto&[u,v,w]:Edges){
        if(st.merge(u,v)){
            sum+=w;
        }
    }
    if(st.count()==1){
        return sum;
    }
    return -1;
}
```

4.4 最小生成树 Prim

```
T prim(int start,int n){
    // {w,u}
    priority_queue<pair<T,int>,vector<pair<T,int>>,
        greater<>>pq;
    vector<char>vis(n,0);
    T sum=0;
    int cnt=0;
    pq.push({0,start});
    while(pq.size()){
        auto [d,u]=pq.top();
        pq.pop();
        if(vis[u])continue;
        vis[u]=1;
        sum+=d;
        cnt++;
        if(cnt==n){
            return sum;
        }
        for(int ei=head[u];ei!=-1;ei=nex[ei]){
            int v=to[ei];
            if(!vis[v]){
                pq.push({weight[ei],v});
            }
        }
    }
    return -1;
}
```

4.5 最短路 Floyd

```
const int N=105;
const int INF=0x3f3f3f3f;
T dis[N][N];
void init(int n){
    for(int i=0;i<n;++i){
        for(int j=0;j<n;++j){
            if(i==j)dis[i][j]=0;
            else dis[i][j]=INF;
        }
    }
}
void floyd(int n){
    for(int k=0;k<n;++k){
        for(int i=0;i<n;++i){
            if(i==k||dis[i][k]==INF)continue;
            for(int j=0;j<n;++j){
                if(j==k||dis[k][j]==INF)continue;
                if(dis[i][j]>dis[i][k]+dis[k][j]){
                    dis[i][j]=dis[i][k]+dis[k][j];
                }
            }
        }
    }
}
bool check(int n){
    // floyd(n);
    for(int i=0;i<n;++i){
        if(dis[i][i]<0){
            return 1; //有负环
        }
    }
    return 0; //无负环
}
```

4.6 最短路 Dijkstral

```
T dis[N];
bool vis[N];
void dijkstra(int start){
    memset(dis,0x3f,sizeof(dis));
    memset(vis,0,sizeof(vis));
    // {w,u}
    priority_queue<pair<T,int>,vector<pair<T,int>>,
        greater<>>pq;
    dis[start]=0;
    pq.push({0,start});
```

```
    while(pq.size()){
        auto [c,u]=pq.top();
        pq.pop();
        if(vis[u])continue;
        vis[u]=1;
        for(int ei=head[u];ei!=-1;ei=nex[ei]){
            int v=to[ei];
            T w=weight[ei];
            T nc=c+w;
            if(dis[v]>nc){
                dis[v]=nc;
                pq.push({nc,v});
            }
        }
    }
}
```

4.7 最短路 SPFA

```
T dis[N];
bool in[N];
void spfa(int start){
    memset(dis,0x3f,sizeof(dis));
    memset(in,0,sizeof(in));
    queue<int>q;
    dis[start]=0;
    q.push(start);
    in[start]=1;
    while(q.size()){
        int u=q.front();
        q.pop();
        in[u]=0;
        for(int ei=head[u];ei!=-1;ei=nex[ei]){
            int v=to[ei];
            T w=weight[ei];
            if(dis[v]>dis[u]+w){
                dis[v]=dis[u]+w;
                if(in[v]==0){
                    q.push(v);
                    in[v]=1;
                }
            }
        }
    }
}
// 检查负环
int cnt[N]; //cnt[u]:从起点到u的最短路上的边数
bool spfa_check(int s, int n){
    memset(dis,0x3f,sizeof(dis));
    memset(cnt,0,sizeof(cnt));
    memset(in,0,sizeof(in));
    queue<int> q;
    q.push(s);
    dis[s]=0;
    cnt[s]=0;
    in[s]=1;
    while(q.size()){
        int u=q.front();
        q.pop();
        in[u]=0;
        for(int ei=head[u];ei!=-1;ei=nex[ei]){
            int v=to[ei];
            T w=weight[ei];
            if(dis[v]>dis[u]+w){
                dis[v]=dis[u]+w;
                cnt[v]=cnt[u]+1;

                if(cnt[v]>=n)return 1; //有负环

                if(in[v]==0){
                    q.push(v);
                    in[v]=1;
                }
            }
        }
    }
    return 0; //无负环
}
```

4.8 LCA

```
const int N = 1e5 + 5;
int depth[N], fa[N][20];
vector<int> adj[N];
//dfs(1,0) 根节点为 1
void dfs(int u,int f){
    depth[u]=depth[f]+1;
    fa[u][0]=f;
    for(int i=1;i<=19;++i){
        fa[u][i]=fa[fa[u][i-1]][i-1];
    }
    for(int v:adj[u]){
        if(v!=f)dfs(v,u);
    }
}
int lca(int u, int v) {
    if(depth[u]<depth[v])swap(u,v);
    for(int i=19;i>=0;--i){
        if(depth[fa[u][i]]>=depth[v]){
            u=fa[u][i];
        }
    }
    if(u==v)return u;
    for(int i=19;i>=0;--i){
        if(fa[u][i]!=fa[v][i]){
            u=fa[u][i];
            v=fa[v][i];
        }
    }
    return fa[u][0];
}
```

5 近期做的一些题（可删）

5.1 轮廓线 DP

```
int n,m;
void solve(){
    if(n*m&1){
        cout<<0<<'\n';
        return;
    }
    vector<vector<vector<ll>>>dp(n+1,vector<vector<ll>>(m+1,vector<ll>(1<<m)));
    dp[0][m][0]=1;
    for(int i=1;i<=n;++i){
        for(int j=1;j<=m;++j){
            int pi=j==1?i-1:i;
            int pj=j==1?m:j-1;
            for(int s=0;s<(1<<m);++s){
                int bit=j-1;
                if((s>>bit)&1){
                    int t=s^(1<<bit);
                    dp[i][j][t]+=dp[pi][pj][s];
                }
                else{
                    if(j<m&&((s>>(bit+1))&1)==0){
                        int t=s|(1<<(bit+1));
                        dp[i][j][t]+=dp[pi][pj][s];
                    }
                    if(i<n){
                        int t=s|(1<<bit);
                        dp[i][j][t]+=dp[pi][pj][s];
                    }
                }
            }
        }
    }
    cout<<dp[n][m][0]<<'\n';
}
```

5.2 互不侵犯

```
int n,K;
vector<int>cols;
void solve(){
    cin>>n>>K;

    for(int col=0;col<(1<<n);++col){
        bool ok=1;
        for(int i=1;ok&&i<n;++i){
            if(((col>>i)&1)&&((col>>(i-1))&1)){
                ok=0;
            }
        }
        if(ok){
            cols.push_back(col);
        }
    }
    int m=cols.size();

    vector dp(n+1,vector<vector<ll>>(m,vector<ll>(K+1)));
    dp[0][0][0]=1;
    for(int i=1;i<=n;++i){
        for(int j=0;j<m;++j){
            int cur=cols[j];
            int cnt=__builtin_popcount(cur);
            for(int k=0;k<m;++k){
                int pre=cols[k];
                if((cur&pre)==0&&(cur&(pre>>1))==0&&(cur&(pre<<1))==0){
                    for(int nums=cnt;nums<=K;nums++){
                        dp[i][j][nums]+=dp[i-1][k][nums-cnt];
                    }
                }
            }
        }
    }

    ll ans=0;
    for(int i=0;i<m;++i){
        ans+=dp[n][i][K];
    }
    cout<<ans<<'\n';
}
```

5.3 分组过桥

```
void solve(){
    int W,n;
    cin>>W>>n;
    vector<int>t(n),w(t);
    for(int i=0;i<n;++i)cin>>t[i]>>w[i];

    vector<int>st(1<<n),sw(1<<n);
    for(int s=1;s<(1<<n);++s){
        int cost=0;
        int weight=0;
        for(int i=0;i<n;++i){
            if((s>>i)&1){
                cost=max(cost,t[i]);
                weight+=w[i];
            }
        }
        st[s]=cost;
        sw[s]=weight;
    }

    vector<int>dp(1<<n,0x3f3f3f3f);
    dp[0]=0;
    for(int s=1;s<(1<<n);++s){
        for(int t=s;t;t=(t-1)&s){
            if(sw[t]>W)continue;
            dp[s]=min(dp[s],st[t]+dp[s^t]);
        }
    }

    cout<<dp[(1<<n)-1]<<'\n';
}
```

5.4 用 t 拼 s, 求最短 t

```
void solve(){
    int L;
    string ch;
    cin>>L>>ch;
    ch=" "+ch;
    vector<int>pi(L+1);
    for(int i=2,j=0;i<=L;++i){
        while(j&&ch[j+1]!=ch[i])j=pi[j];
        if(ch[j+1]==ch[i])j++;
        pi[i]=j;
    }
    cout<<L-pi[L];
}
```

5.5 能量项链

```
const int N=205;
int n;
int mem[N][N];
int head[N];
int tail[N];

int dfs(int L,int R){
    if(mem[L][R]){
        return mem[L][R];
    }
    if(L==R){
        return 0;
    }
    int res=INT_MIN;
    for(int k=L;k<R;++k){
        res=max(res,dfs(L,k)+dfs(k+1,R)+head[L]*tail[k
            ]*tail[R]);
    }
    return mem[L][R]=res;
}

void solve(){
    cin>>n;
    for(int i=0;i<n;++i){
        cin>>head[i];
        head[i+n]=head[i];
    }
    for(int i=0;i<n;++i){
        tail[i]=i==n-1?head[0]:head[i+1];
        tail[i+n]=tail[i];
    }
    int ans=INT_MIN;
    for(int L=0;L<n;++L){
        int R=L+n-1;
        ans=max(ans,dfs(L,R));
    }
    cout<<ans;
}
```

5.6 编辑距离

```
const int N=2005;
string s,t;
int mem[N][N];
int dfs(int i,int j){
    if(mem[i][j]!=-1){
        return mem[i][j];
    }
    if(i==s.size()){
        return mem[i][j]=t.size()-j;
    }
    if(j==t.size()){
        return mem[i][j]=s.size()-i;
    }
    if(s[i]==t[j]){
        return mem[i][j]=dfs(i+1,j+1);
    }
    return mem[i][j]=min({dfs(i+1,j),dfs(i,j+1),dfs(i
        +1,j+1)})+1;
}
void solve(){
    cin>>s>>t;
    memset(mem,-1,sizeof(mem));
    cout<<dfs(0,0);
}
```

5.7 铁盘整理 IDA*

```
int n;
ull re(ull u,int len){
    ull v=u;
    int l=0,r=len;
    while(l<r){
        int a=15&(u>>(l<<2));
        int b=15&(u>>(r<<2));
        v&=~(15ULL<<(l<<2));
        v&=~(15ULL<<(r<<2));
        v|=(ull)a<<(r<<2);
        v|=(ull)b<<(l<<2);
        l++;
        r--;
    }
    return v;
}
int f(ull s){
    int cnt=0;
    for(int i=1;i<n;++i){
        int a=15&s>>(i<<2);
        int b=15&s>>((i-1)<<2);
        if(abs(a-b)!=1)cnt++;
    }
    int back=15&s>>((n-1)<<2);
    if(back!=n-1)cnt++;
    return cnt;
}
bool dfs(int cnt,int limit,ull u,ull p){
    int h=f(u);
    if(h==0)return 1;
    if(cnt+h>limit)return 0;
    for(int len=1;len<n;++len){
```

```
        ull v=re(u,len);
        if(v==p)continue;
        if(dfs(cnt+1,limit,v,u))return 1;
    }
    return 0;
}
void solve(){
    cin>>n;
    vector<pair<int,int>>id(n);
    for(int i=0;i<n;++i){
        cin>>id[i].first;
        id[i].second=i;
    }
    sort(id.begin(),id.end());
    ull s=0;
    for(ull i=0;i<n;++i){
        s|=(ull)id[i].second<<(i<<2);
    }
    int limit=f(s);
    while(limit&&!dfs(0,limit,s,-1)){
        limit++;
    }
    cout<<limit<<'\n';
}
```

5.8 流星雨

```
const int inf=1e5;
const int N=305;
int M[N][N];
int mo[]{-1,0,1,0,-1};

void solve(){
    for(int i=0;i<N;++i){
        for(int j=0;j<N;++j){
            M[i][j]=inf;
        }
    }
    int m;
    cin>>m;
    while(m--){
        int x,y,t;
        cin>>x>>y>>t;
        M[x][y]=min(M[x][y],t);
        for(int i=0;i<4;++i){
            int nx=x+mo[i];
            int ny=y+mo[i+1];
            if(nx<0||ny<0||nx>=N||ny>=N)continue;
            M[nx][ny]=min(M[nx][ny],t);
        }

    }
    if(M[0][0]==0){
        cout<<-1<<'\n';
        return;
    }
    queue<pair<int,int>>q;
    q.push({0,0});
    int t=0;
    while(q.size()){
        int siz=q.size();
        while(siz--){
            auto [x,y]=q.front();
            q.pop();
            for(int i=0;i<4;++i){
                int nx=x+mo[i];
                int ny=y+mo[i+1];
                if(nx<0||ny<0||nx>=N||ny>=N)continue;
                if(M[nx][ny]<=t+1)continue;
                if(M[nx][ny]==inf){
                    cout<<t+1<<'\n';
                    return;
                }
                q.push({nx,ny});
                M[nx][ny]=0;
            }
        }
        t++;
    }
    cout<<-1<<'\n';
}
```

5.9 青蛙的约会

```
ll ex_gcd(ll a,ll b,ll &x,ll &y){
    if(b==0){
        x=1,y=0;
        return a;
    }
    ll g=ex_gcd(b,a%b,y,x);
    y-=a/b*x;
    return g;
}

ll n,m,vn,vm,L,x,y,a,b,c;

void solve(){
    cin>>n>>m>>vn>>vm>>L;
    if(vn==vm){
        cout<<"Impossible";
        return;
    }
    if(n>m){
        if(vm>vn){
            a=vm-vn;
            c=n-m;
        }
        else{
            a=vn-vm;
            c=L-(n-m);
        }
    }
    else{
        if(vn>vm){
            a=vn-vm;
            c=m-n;
        }
        else{
            a=vm-vn;
            c=L-(m-n);
        }
    }
```

```
    b=L;
    ll g=ex_gcd(a,b,x,y);
    if(c%g!=0){
        cout<<"Impossible";
    }
    else{
        ll kx=b/g;
        x*=c/g;
        if(x>0){
            ll k=(x-1)/kx;
            x-=k*kx;
        }
        else{
            ll k=(1-x+kx-1)/kx;
            x+=k*kx;
        }
        cout<<x;
    }
}
```

5.10 生成字符串（卡特兰数）

```
void solve(){
    int n,m;
    cin>>n>>m;
    if(n<m){
        cout<<0;
        return;
    }
    cout<<(C(n+m,m)-C(n+m,m-1)+MOD)%MOD;
}
```

5.11 反向建边求返程

```
void solve(){
    cin>>n>>m;
    for(int i=0;i<m;++i){
        cin>>u>>v>>w;
        E.push_back({u,v,w});
    }

    ll ans=0;

    init(n);
    for(auto [u,v,w]:E){
        addEdge(u,v,w);
    }
    dijkstra(1);
    for(int i=2;i<=n;++i){
        ans+=dis[i];
    }

    init(n);
    for(auto [u,v,w]:E){
        addEdge(v,u,w);
    }
    dijkstra(1);
    for(int i=2;i<=n;++i){
        ans+=dis[i];
    }

    cout<<ans;
}
```

5.12 网格图连通

```
void solve(){
    int n,m;
    cin>>n>>m;
    vector<ull>a(n),b(m);
    for(int i=0;i<n;++i)cin>>a[i];
    for(int i=0;i<m;++i)cin>>b[i];
    sort(a.begin(),a.end());
    sort(b.begin(),b.end());
    vector<ull>pre_b(m+1);
    for(int i=0;i<m;++i){
        pre_b[i+1]=pre_b[i]+b[i];
    }
    ull ans=a[0]*(m-1)+b[0]*(n-1);
    for(int i=1;i<n;++i){
        int j=lower_bound(b.begin()+1,b.end(),a[i])-b.
            begin();
        ans+=pre_b[j]-pre_b[1];
        ans+=a[i]*(m-j);
    }
    cout<<ans;
}
```

5.13 二分判连通

```
const int N=405*405;
int mo[]{-1,0,1,0,-1};
vector<pair<ll,int>>gra[N];
int t[N];
ll dis[N];
int n,m,k,money;
bool check(int limit){
    for(int i=0;i<n*m;++i)dis[i]=0x3f3f3f3f3f3f3f3f;
    priority_queue<pair<ll,int>,vector<pair<ll,int>>,
        greater<>>pq;
    pq.push({0,0});
    dis[0]=0;
    while(pq.size()){
        auto [c,u]=pq.top();
        pq.pop();

        if(c>dis[u])continue;

        for(auto [w,v]:gra[u]){
            if(t[v]>limit)continue;
            ll nc=c+w;
            if(nc<dis[v]){
                dis[v]=nc;
                pq.push({nc,v});
            }
        }
        int x=u/m;
        int y=u%m;
        for(int i=0;i<4;++i){
            int nx=x+mo[i];
            int ny=y+mo[i+1];
            if(nx<0||ny<0||nx>=n||ny>=m)continue;
```

```
            int v=nx*m+ny;
            if(t[v]>limit)continue;
            if(c<dis[v]){
                dis[v]=c;
                pq.push({c,v});
            }
        }
    }
    return dis[n*m-1]<=money;
}
void solve(){
    cin>>n>>m>>money>>k;
    for(int i=0;i<n;++i){
        for(int j=0;j<m;++j){
            cin>>t[i*m+j];
            gra[i*m+j].clear();
        }
    }
    while(k--){
        int x,y,p,q,w;
        cin>>x>>y>>p>>q>>w;
        x--,y--,p--,q--;
        gra[x*m+y].push_back({w,p*m+q});
    }
    int l=0,r=1e9,mid,ans=-1;
    while(l<=r){
        mid=(l+r)>>1;
        if(check(mid))ans=mid,r=mid-1;
        else l=mid+1;
    }
    cout<<ans<<'\n';
}
```

5.14 最大三段和

```
void solve(){
    int n;
    cin>>n;
    vector<int>a(n);
    for(int i=0;i<n;++i)cin>>a[i];

    vector<int>pre(n+1,-inf);
    int sum=-inf;
    for(int i=0;i<n;++i){
        sum=max(sum+a[i],a[i]);
        pre[i+1]=max(pre[i],sum);
    }

    vector<int>suf(n+1,-inf);
    sum=-inf;
    for(int i=n-1;i;--i){
        sum=max(sum+a[i],a[i]);
        suf[i-1]=max(suf[i],sum);
    }
    int ans=INT_MIN;
    for(int i=2;i<n-2;++i){
        ans=max(ans,pre[i-1]+a[i]+suf[i+1]);
    }
    cout<<ans<<'\n';
}
```

5.15 质数表跳跃

```
int val[]{
    0,
    2,3,5,7,11,
    13,17,19,23,29,
    31,37,41,43,47,
    53,59,61,67,71,
    73,79,83,89,97,
    101,103,107,109,113,
    127
};

int need[114];
int maxp[130];

void init(){
    for(int i=0;i<=30;++i){
        need[val[i]]=val[i+1]-val[i];
    }
    for(int i=0;i<130;++i){
        for(int v:val){
            if(v<=i){
                maxp[i]=v;
            }
            else{
                break;
            }
        }
    }
}
int f(vector<int>&a){
    int n=a.size()>>1;

    vector nex(a.size()+1,vector<int>(11,0x3f3f3f3f));
    for(int i=a.size()-1;i>=0;--i){
        for(int j=10;j>=0;--j){
            if(a[i]>=j){
                nex[i][j]=i;
            }
            else{
                nex[i][j]=nex[i+1][j];
            }
        }
    }

    int ans=0;
    for(int st=0;st<n;++st){
        int end=st+n-1;
        int cur_val=0;
        int cur_pos=st;
        while(cur_pos<=end){
            int tar=need[cur_val];
            if(tar>10){
                break;
            }
            int to=nex[cur_pos][tar];
            if(to>end){
                break;
            }
            cur_val=maxp[cur_val+a[to]];
```

```
            if(cur_val==113){
                return 113;
            }
            cur_pos=to+1;
        }
        ans=max(ans,cur_val);
    }
    return ans;
}
void solve(){
    int n;
    cin>>n;
    vector<int>a(n*2),b(n*2);
    for(int i=0;i<n;++i){
        cin>>a[i];
        a[i+n]=a[i];
        b[2*n-1-i]=b[n-1-i]=a[i];
    }
    cout<<max(f(a),f(b))<<'\n';
}
```

5.16 任意段和不整除

```
void solve(){
    int n,k;
    cin>>n>>k;
    if(n>k-1){
        cout<<-1<<'\n';
        return;
    }
    set<int>st;
    for(int i=2;i<k;++i){
        st.insert(i);
    }
    int sum=1;
    vector<int>a(n,1);
    for(int i=1;i<n;++i){
        int L=(sum+a[i-1]+1)%k;
        auto it=st.lower_bound(L);
        if(it==st.end()){
            it=st.begin();
            a[i]=a[i-1]+(k+*it-L+1);
        }
        else{
            a[i]=a[i-1]+(*it-L+1);
        }
        sum=(sum+a[i])%k;
        st.erase(it);
    }
    for(int i=0;i<n;++i){
        cout<<a[i]<<(i==n-1?'\n':' ');
    }
}
```